Project Report: Image Processing System

S3 → EC2 → S3 Image Processing Using an AWS IAM Role

Course / Subject: Cloud Computing / AWS	**Name:** Amaranath K
AWS Region: Europe(Ireland) eu-west-1	**Date:** October 2026

1. Introduction

Amazon Web Services (AWS) provides secure and scalable cloud infrastructure for building automated workloads. This report documents a hands-on lab in which images are uploaded to an Amazon S3 bucket (input/ folder), downloaded and processed on an Amazon EC2 instance using Python and the Pillow library (resized and compressed), and then uploaded back to the same bucket (output/ folder).

Access between EC2 and S3 is granted through an IAM Role attached to the instance, which removes the need for hardcoded AWS credentials in the script or on the server.

Architecture: S3 (input/) → EC2 downloads image → Python + Pillow processing → S3 (output/)

Services used: Amazon S3, Amazon EC2, IAM Role, AWS CLI

2. Step-by-Step Procedure

Step 1: Creating the S3 Bucket

1. Navigate to the Amazon S3 Console and click **Create bucket**.
2. Set the Bucket Name to **image-processing-amarnath** (globally unique).
3. Choose the same Region as the EC2 instance.
4. Click **Create bucket**.
5. Inside the bucket, create two folders: **input/** and **output/**.

Step 2: Creating an IAM Role for EC2

1. Open the IAM Console, navigate to **Roles**, and click **Create role**.
2. Select **AWS service** as the trusted entity type and choose **EC2** as the use case.
3. Attach the permission policy **AmazonS3FullAccess** (for training purposes).
4. Name the role **EC2-ImageProcess-S3Role** and complete the role creation.

Step 3: Launching the EC2 Instance

1. Navigate to the EC2 Console and click **Launch instance**.
2. Select **Amazon Linux 2 / Amazon Linux 2023** as the AMI and **t2.micro** as the instance type.
3. Under **IAM instance profile**, attach **EC2-ImageProcess-S3Role**.
4. Configure the Security Group to allow **SSH (Port 22)** from your IP only.
5. Launch the instance.

Step 4: Connecting to the EC2 Instance

1. Connect to the instance over SSH:

```
ssh ec2-user@<EC2_PUBLIC_IP>
```

Step 5: Installing Required Packages

1. Update the system and install Python, pip and the AWS CLI:

```
sudo yum update -y
sudo yum install -y python3 pip awscli
```

2. Verify the installed versions:

```
python3 --version
aws --version
```

Step 6: Installing the Pillow and Boto3 Libraries

1. Install Pillow (image processing library) and Boto3 (AWS SDK for Python):

```
pip3 install pillow boto3
```

The terminal output in Figure 1 shows Boto3 1.42.97 and its dependencies (botocore, s3transfer) being installed successfully.

Step 7: Creating the Image Processing Script

1. Create the working folder and open a new script file:

```
mkdir image-lab && cd image-lab
vi process_images.py
```

2. Paste the following code (bucket name updated to **image-processing-amarnath**):

```
import boto3
from PIL import Image

BUCKET_NAME = "image-processing-amarnath"
INPUT_PREFIX = "Input/"
OUTPUT_PREFIX = "Output/"

s3 = boto3.client("s3")

def process_image(key):
    filename = key.split("/")[-1]
    download_path = f"/tmp/{filename}"
    output_path = f"/tmp/processed-{filename}"

    print(f"Downloading: {key}")
    s3.download_file(BUCKET_NAME, key, download_path)

    print("Processing image...")
    img = Image.open(download_path)

    # Resize image
    img = img.resize((400, 400))

    # Save compressed image
    img.save(output_path, optimize=True, quality=60)
```

```
    output_key = OUTPUT_PREFIX + "processed-" + filename

    print(f"Uploading processed image to: {output_key}")
    s3.upload_file(output_path, BUCKET_NAME, output_key)

    print("Done\n")

def main():
    print("Listing input images...")
    response = s3.list_objects_v2(Bucket=BUCKET_NAME, Prefix=INPUT_PREFIX)

    if "Contents" not in response:
        print("No images found in input/")
        return

    for obj in response["Contents"]:
        key = obj["Key"]
        if key.endswith("/"):
            continue
        process_image(key)

if __name__ == "__main__":
    main()
```

How the script works: it lists all objects under input/, downloads each image to /tmp, resizes it to 400 × 400 pixels, saves it with optimization at quality 60 (compression), and uploads the result to output/ with a processed- prefix.

Step 8: Uploading Test Images to S3

1. Go to **S3** → **image-processing-amarnath** → **input/**.
2. Upload the test images. one image were used: whatsapp image

Step 9: Running the Processing Script

1. Run the script from the image-lab directory:

```
python3 process_images.py
```

2. Confirm that each image is downloaded, processed and uploaded, as shown in Figure 1.

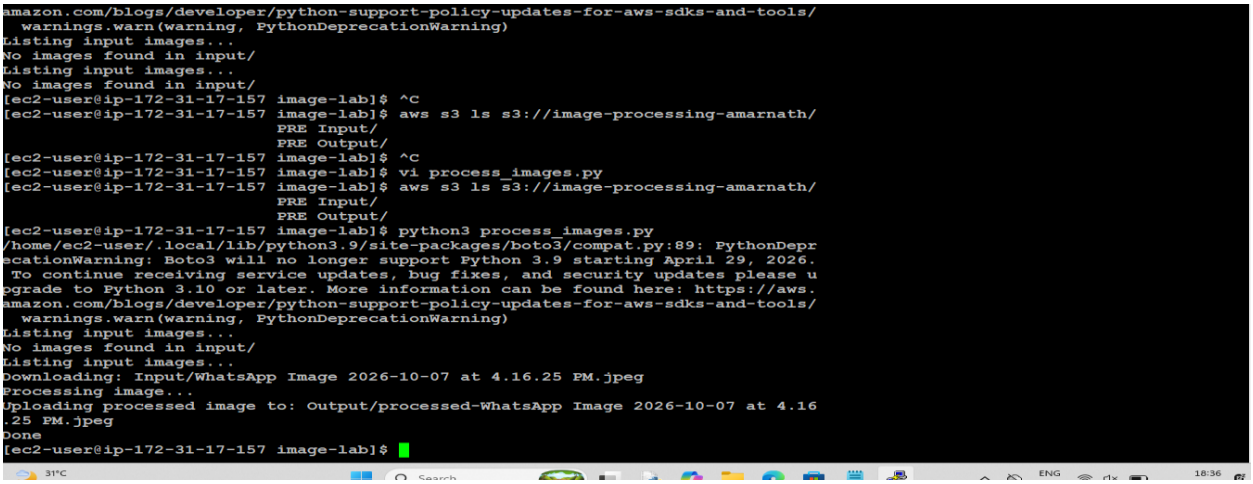


Figure 1: Boto3 installation and execution of process_images.py

The output confirms that both images were handled: **in Input/WhatsApp Image** was uploaded as **output/processed.** The Python 3.9 deprecation warning and the pip root-user warning shown are informational and did not affect the result.

Step 10: Automating with a Cron Job (Optional)

1. Open the cron editor:

```
crontab -e
```

2. Add the following line to run the script every 5 minutes and log its output:

```
*/5 * * * * /usr/bin/python3 /home/ec2-user/image-lab/process_images.py >> /home/ec2-user/image-lab/log.txt 2>&1
```

Step 11: Verifying IAM Role Access

1. Run the AWS CLI listing commands inside the EC2 instance:

```
aws s3 ls s3://image-processing-amarnath/input/
aws s3 ls s3://image-processing-amarnath/output/
```

2. Confirm that both folders are listed without entering any AWS access keys, which validates that the attached IAM Role is active.

```
Done
[ec2-user@ip-172-31-17-157 image-lab]$ aws s3 ls s3://image-processing-amarnath/output/
[ec2-user@ip-172-31-17-157 image-lab]$ aws s3 ls s3://image-processing-amarnath/input/
[ec2-user@ip-172-31-17-157 image-lab]$ python3 process_images.py
/home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer suppo
rt Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgra
de to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-poli
cy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
Listing input images...
No images found in input/
Listing input images...
Downloading: Input/WhatsApp Image 2026-10-07 at 4.16.25 PM.jpeg
Processing image...
Uploading processed image to: Output/processed-WhatsApp Image 2026-10-07 at 4.16.25 PM.jpeg
Done
[ec2-user@ip-172-31-17-157 image-lab]$ aws s3 ls s3://image-processing-amarnath/output/
[ec2-user@ip-172-31-17-157 image-lab]$ python3 process_images.py
/home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer suppo
rt Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgra
de to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-poli
cy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
Listing input images...
No images found in input/
Listing input images...
Downloading: Input/WhatsApp Image 2026-10-07 at 4.16.25 PM.jpeg
Processing image...
Uploading processed image to: Output/processed-WhatsApp Image 2026-10-07 at 4.16.25 PM.jpeg
Done
[ec2-user@ip-172-31-17-157 image-lab]$
```

Figure 2: Listing input/ and output/ folders using the AWS CLI with the IAM Role

3. Verification & Summary

S3 Bucket	image-processing-amarnath (folders: input/, output/)
Attached IAM Role	EC2-ImageProcess-S3Role (AmazonS3FullAccess)
Runtime Environment	Python 3.9 with Boto3 and Pillow on Amazon Linux (EC2 t2.micro)
Network & Security	SSH (Port 22) allowed from own IP only
Execution Status	Script processed 2 images from input/, saved results to output/, and access was verified via the AWS CLI without credentials

Result: File sizes before and after processing

Input file	Original size	Output file	Processed size	Reduction
photo.jpg	73,476 bytes	Processed-whatsapp-images.jpg	18,966 bytes	about 74%

Sizes are taken from the S3 listing in Figure 2.

4. Conclusion

The image processing system was successfully built and executed on an AWS EC2 instance. Images placed in the S3 input/ folder were downloaded by the EC2 instance, resized to 400 × 400 pixels, compressed using Pillow, and uploaded to the output/ folder with a processed- prefix, reducing file sizes in the process.

By attaching an IAM Role to the EC2 instance, the script and the AWS CLI authenticated with Amazon S3 using temporary credentials, eliminating the security risk of hardcoded access keys. The same approach can be extended with a cron job to process new images automatically at regular intervals.